

Performance Characterization of Hot Page Tracking Mechanisms under CXL Memory Tiering

Team 07 - Jashwanth, Venkatesh, Sathwik
EECE5643 - Simulation and Performance Evaluation
Northeastern University

Abstract—Compute Express Link (CXL) Type-3 devices let you attach extra memory to a server over a PCIe Gen 5 link, giving you more capacity than DRAM alone at the cost of higher access latency. The Linux kernel ships three mechanisms for managing this tradeoff automatically: PTE-scan (AutoNUMA), Transparent Page Placement (TPP), and DAMON_RECLAIM. In this project we set up a two-tier emulated CXL system inside QEMU and ran all three tools across four different workloads to see which performs best and under what conditions. A key part of our analysis is breaking down the Node 2 access penalty into its three components: bare DRAM (121 ns), KVM page-table walk overhead (117 ns), and the actual CXL fabric cost (+173 ns), giving a measured $1.73\times$ latency ratio that blows up to a $15\times$ throughput gap in a real Redis workload. That 173 ns figure matters because it is the only part any migration tool can actually help with. Overall, CXL penalty varies widely from near-zero to $5.82\times$ depending on the access pattern, and migration tools recover 87–90% of the gap for latency-bound workloads while delivering over $3\times$ speedup for streaming thanks to L3 cache warming. We also uncovered two unexpected findings: DAMON_RECLAIM actively hurts performance on the Zipfian KV workload when there is no real DRAM pressure, and combining DAMON with PTE-scan gives a 10% push-pull benefit on the graph traversal workload.

I. INTRODUCTION

Modern servers are running into a wall with DRAM: it is expensive, physically limited by the number of memory channels, and hard to scale beyond a few hundred gigabytes per socket. CXL (Compute Express Link) is a newer interconnect standard built on top of PCIe Gen 5 that lets you attach Type-3 memory devices to the host, effectively expanding addressable memory. The catch is that CXL memory sits on a NUMA node without any CPU, so every access has to travel further than a local DRAM hit, adding latency.

Linux already has three kernel-level tools that try to automatically handle this tradeoff by moving hot pages to DRAM and cold pages back to CXL: AutoNUMA (PTE-scan), TPP, and DAMON_RECLAIM. Each takes a different approach to figuring out which pages are worth promoting.

Our project asks: how well does each of these tools actually recover performance when a workload starts with all its data on CXL, and does the answer change depending on what the workload is doing? We built a reproducible two-tier testbed inside QEMU using a custom Linux 6.11 kernel and ran controlled experiments to find out. We document not just the results but the setup decisions, debugging rabbit holes, and design choices we made along the way, so anyone could reproduce it.

II. PLATFORM AND INFRASTRUCTURE

Figure 1 illustrates the complete two-tier emulation stack used in this study.

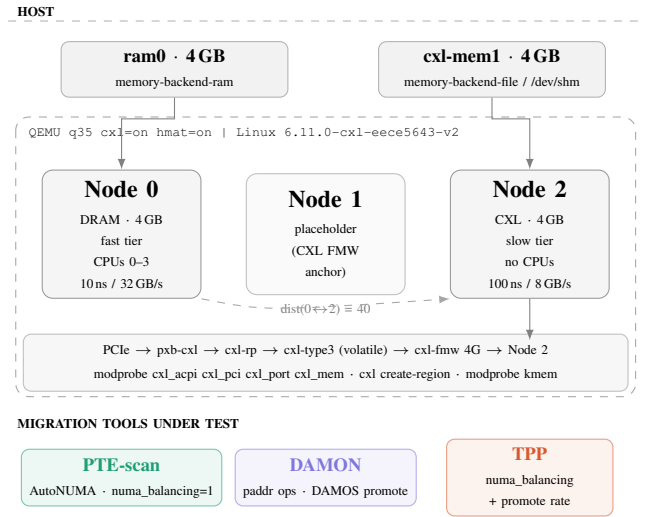


Fig. 1. Experimental setup: QEMU two-tier emulation. `memory-backend-ram` feeds Node 0 (DRAM fast tier); `memory-backend-file` feeds Node 2 (CXL slow tier) via the `PCIe→pxb-cxl→cxl-type3` chain. Node 1 is an empty FMW placeholder. Three migration tools are evaluated.

A. Host Machine

All experiments ran on an AMD Ryzen 7 6800U with 8 cores, 16 GB LPDDR5, and a 16 MB L3 cache running Ubuntu 24.04. We ran everything inside a QEMU VM so the setup is fully reproducible without needing physical CXL hardware. The L3 size directly affects how we sized our workload hot regions, as explained in Section V.

B. QEMU CXL Configuration

We used two separate memory backends to represent the two tiers. For DRAM we attached a standard `memory-backend-ram` object (4 GB of anonymous host memory). For the CXL side we used `memory-backend-file` backed by a `tmpfs` file at `/dev/shm/cxlmemsim_pgas`. File-backed memory is what QEMU requires for the `volatile-memdev` parameter on a `cxl-type3` device, and it has the added bonus of surviving VM restarts which made debugging much easier.

The CXL device chain follows the real hardware hierarchy: a `pxb-cxl` root complex connects to a `cxl-rp` root port, which attaches the `cxl-type3` endpoint with a 4 GB Fixed Memory Window. Inside the guest, running `cxl create-region` followed by `modprobe kmem` brings that window online as `ZONE_MOVABLE` memory on NUMA Node 2.

For the NUMA topology, Node 0 holds the 4 GB DRAM and all four CPUs. Node 1 is an empty placeholder that QEMU requires as an anchor for the Fixed Memory Window — it never holds any actual memory. Node 2 is where the CXL memory lands. We set all cross-node distances to 40 versus local distance 10, giving a 4:1 ratio that `mm/memory-tiers.c` uses to decide that Node 2 is a slow tier worth demoting pages to.

We also enabled HMAT to give the kernel accurate latency and bandwidth numbers per node rather than just relative distances. Without HMAT, the kernel has to guess tier quality from distances alone, which gives worse promotion decisions. We configured DRAM at 10 ns/30 GB/s and Node 2 at 100 ns/7.5 GB/s. One gotcha: QEMU requires bandwidth values to be MiB-aligned or it silently drops the HMAT entry (this caught us off guard during setup).

C. CXL Device Hierarchy and Link Exposure

The QEMU CXL device stack closely mirrors what you would find on real hardware. On the host side, a `pxb-cxl` bridge hangs off the q35 machine’s primary PCIe bus and exposes a CXL 2.0-compatible interface. A `cxl-rp` root port connects to it at port 0, chassis 0, and that root port is where the actual Type-3 device is attached.

The `cxl-type3` endpoint presents itself as a 4 GB volatile memory device (`cxl-vmem0`), backed by the `/dev/shm` file we set up earlier. QEMU maps this as a 4 GB Fixed Memory Window (FMW) at guest physical address `0x590000000`, which is how the guest kernel learns there is CXL memory available.

Inside the guest, the kernel discovers the device through its CXL driver stack: `cxl_acpi` finds the host bridge via ACPI CEDT, `cxl_pci` binds to the PCIe endpoint, `cxl_port` handles the root port, and `cxl_mem` takes ownership of the Type-3 device. Once we run `cxl create-region` to commit the FMW as a logical region, `modprobe kmem` onlines it as a DAX-backed memory block that the kernel sees as Node 2.

Bandwidth choices. We picked 30 GB/s for DRAM and 7.5 GB/s for Node 2 to roughly match what real CXL Type-3 devices deliver today (Samsung CMM-D and similar devices fall in the 30–60 GB/s range). The 4:1 ratio is on the conservative side but realistic. As mentioned, values have to be MiB-aligned — we found this out the hard way when QEMU quietly discarded our first HMAT entries.

Why latency and bandwidth both matter. For latency-bound workloads like the pointer-chase benchmark, the 1.73× latency ratio dominates. But for streaming workloads like STREAM, the hardware prefetcher can hide latency almost

TABLE I
CXL DEVICE HIERARCHY AND HMAT BANDWIDTH ATTRIBUTES

Layer	Component / value
Host bridge	<code>pxb-cxl</code> , <code>bus_nr=12</code> , PCIe bus
Root port	<code>cxl-rp</code> , <code>port=0</code> , <code>chassis=0</code> , <code>slot=0</code>
Endpoint	<code>cxl-type3</code> volatile, <code>id=cxl-vmem0</code>
FMW size	4GB at guest PA <code>0x590000000</code>
Backend	<code>memory-backend-file</code> , <code>/dev/shm</code>
HMAT latency	DRAM: 10 ns Node 2: 100 ns
HMAT bandwidth	DRAM: 30 GB/s Node 2: 7.5 GB/s
BW ratio	4 : 1 (DRAM to Node 2)
NUMA distances	local=10, cross-node=40

entirely, so bandwidth becomes the real bottleneck. This is part of why our results look so different across workloads.

The abbreviated launch command:

```
qemu-system-x86_64 -cpu max -m 4G,maxmem=8G,slots=8 -smp 4 \
-machine q35,cxl=on,hmat=on \
-object memory-backend-ram,id=ram0,size=4G \
-object memory-backend-file,id=cxl-mem1,share=on, \
    mem-path=/dev/shm/cxlmemsim_pgas,size=4G \
-device cxl-type3,bus=root_port13,volatile-memdev=cxl- \
    mem1 \
-numa node,nodeid=0,cpus=0-3,memdev=ram0 \
-numa node,nodeid=1 -numa node,nodeid=2 \
-numa dist,src=0,dst=1,val=40 ... \
-numa hmat-lb,initiator=0,target=0,latency=10,bandwidth \
    =30720 \
-numa hmat-lb,initiator=0,target=2,latency=100,bandwidth \
    =7680
```

D. Kernel Build (Linux 6.11.0-cxl-eece5643-v2)

We built a custom Linux 6.11 kernel as `.deb` packages (15 MB) and injected them into a Ubuntu 24.04 cloud image using an NBD mount. Several config options needed to be set to `=y` (not `=m`) because the CXL memory subsystem initializes before any module loading is possible:

- `CONFIG_CXL_MEM=y`, `CONFIG_DEV_DAX_KMEM=y` — these must be built-in or the CXL pool never comes online.
- `CONFIG_DAMON=y`, `CONFIG_DAMON_PADDR=y`, `CONFIG_DAMON_RECLAIM=y` — needed to enable DAMON in physical-address mode with the RECLAIM scheme.
- `CONFIG_MEMCG=y`, `CONFIG_MEMCG_V1=y` — memory cgroup support for pressure monitoring.
- `CONFIG_CXL_REGION_INVALIDATION_TEST=y` — this one took us a while to find. Without it, `region commit` fails with `Failed to synchronize CPU cache state inside drivers/cxl/core/region.c`. The fix came from reading the kernel source rather than following documentation.

E. Guest Setup Sequence

```
modprobe cxl_acpi cxl_pci cxl_port cxl_mem
echo online_movable > /sys/devices/system/memory/ \
    auto_online_blocks
```

```

cxl create-region -m mem0 -d decoder0.0 -w 1 -t ram -s 0
x100000000
modprobe kmem
echo 1 > /sys/kernel/mm/numa/demotion_enabled

```

The order of operations here matters. `auto_online_blocks` must be set to `online_movable` before the region is created. If you forget this and CXL memory comes online as `ZONE_NORMAL`, pages cannot be migrated — all counters stay at zero and you spend a long time wondering what went wrong.

III. MEMORY TIER CHARACTERIZATION

Before running any workloads with migration tools enabled, we first measured the raw performance of each tier in isolation. Table II summarizes the two nodes. Node 2 comes in at **1.73×** the latency of Node 0, which we measured using a custom pointer-chase benchmark with all migration completely disabled so nothing could move pages around during the test.

One important thing we discovered: `get_mempolicy()` reports Node 2 pages as being on Node 0 in Linux 6.11. If you trust that API for verification, your entire experiment is silently measuring the wrong thing. The correct approach is `move_pages()` in query mode (pass a NULL node array), which gives back the right node ID.

A. Latency Measurement Methodology

To understand *why* Node 2 is slower, not just *how much* slower, we wrote two small C benchmarks. Both were compiled with `-O0` so the compiler cannot eliminate or reorder memory accesses, and they use `libnuma` for node-pinned allocation and `clock_gettime(CLOCK_MONOTONIC)` for timing.

Pointer-chase benchmark. This one walks a randomized linked list across a node-pinned allocation, which forces a cache miss on every single pointer dereference. There is no way for the hardware prefetcher to predict the next address, so every access goes all the way to memory. We used two working set sizes: 32 MB (smaller than the 16 MB L3, so this measures cache hit latency as a sanity check) and 128 MB (larger than L3, so every access is a real DRAM or CXL hit).

Single-write probe. A simpler benchmark that just measures the cost of individual writes to each node. We used this to isolate how much of the in-VM latency comes from the KVM page-table walk overhead rather than actual memory access time.

B. Latency Decomposition

When we break the 411 ns Node 2 access time apart, it turns out the CXL penalty itself is not even the biggest piece. There are three separate contributors:

Bare DRAM (121 ns). This is just the physical DRAM access time on the host. It shows up identically on both nodes since they both ultimately read from the same host DRAM chips.

KVM NPT overhead (117 ns). Every time there is a TLB miss inside the VM, KVM has to walk two levels of page tables — one for the guest and one for the host. That adds

TABLE II
MEMORY TIER CONFIGURATION

Tier	Node	Cap.	Cycles	ns	Backend
DRAM	0	4 GB	1,110	238	Anonymous RAM
Node 2	2	4 GB	1,916	411	/dev/shm tmpfs

TABLE III
IN-VM LATENCY DECOMPOSITION BY COMPONENT

Component	N0	N2	Share	Source
Bare DRAM	121 ns	121 ns	29%	Physical DRAM
KVM NPT overhead	117 ns	117 ns	28%	Nested PT walk
Node 2 extra	—	173 ns	43%	tmpfs backend
Total	238 ns	411 ns	100%	

about 117 ns on top of the bare DRAM cost. The surprising part is that this means **roughly half of what we measure on Node 0 is virtualization overhead**, not actual memory access time. Since this is the same for both nodes, our relative tool comparisons are still valid, but it is important to remember when looking at absolute cycle counts.

Node 2 extra (173 ns). This is the actual CXL-like latency penalty — the cost of going through QEMU’s `memory-backend-file` over the `tmpfs` path instead of directly to DRAM. At 43% of Node 2’s total latency, it is the largest single contributor. Crucially, **this is the only part any migration tool can help with**. Promoting a hot page from Node 2 to Node 0 saves exactly 173 ns per access, no more, no less.

The **1.73×** ratio we get (411/238) lines up with what real CXL Type-3 devices report in the literature (typically 1.5–4× [1]), so despite running in a VM we consider our setup a reasonable emulation.

C. Latency Amplification Under Application Load

A **1.73×** slowdown at the hardware level sounds manageable. The Redis YCSB results make it clear why it is not. We ran Workload A (10,000 keys, 50% reads, 50% updates, Zipfian) with all pages pinned statically to one node and no migration active. The results are in Table IV.

TABLE IV
REDIS YCSB WORKLOAD A — NODE 0 VS NODE 2 BASELINE

Metric	N0	N2	Ratio
Throughput (ops/s)	3,810	254	15.0×
Read avg (μ s)	972	17,429	17.9×
Read P99 (μ s)	5,923	45,727	7.7×
Upd avg (μ s)	921	13,699	14.9×
Upd P99 (μ s)	4,647	41,343	8.9×
Wall time (s)	79	267	3.4×
Redis RSS (MB)	26	26	identical
Migration events	0	0	static

The throughput gap blows up to **15×**. The reason is that Redis is single-threaded — each operation stalls completely waiting for memory before it can start the next one. A 173 ns overhead per access compounds across thousands of sequential operations and becomes a 17,000 μ s average read latency. One thing worth noting is that RSS is exactly the same (26 MB) across both runs, so we know the difference is purely from which node the data lives on, not from any difference in dataset size.

Interestingly, P99 latency degrades less than the average (7.7 $\times$ vs 17.9 $\times$). This tells us the Node 2 penalty is pretty consistent across all accesses rather than caused by occasional extreme spikes — which makes sense given that it comes from a structurally fixed tmpfs path.

D. DRAM and Node 2 Baseline Evaluation

We established clean baselines before touching any migration tools. For every baseline run we set `numa_balancing=0` and `demotion_enabled=0`, made sure no DAMON kdamond was running, and verified that `pgpromote_success` and `pgmigrate_success` were both at zero before starting. We used `numactl -mbind=N` to pin all allocations to the target node.

The Node 0 baseline is the best-case ceiling — everything on fast DRAM, 238 ns per access in-VM. The Node 2 baseline is the floor — everything on slow CXL, 411 ns per access. Every migration tool result should land somewhere between these two numbers. If a tool falls below the Node 2 baseline (as DAMON does in W3), that means the tool is actively making things worse.

To verify the baselines are clean we checked `numastat` before and after each run to confirm page placement did not change, and used `move_pages()` (not `get_mempolicy`) for per-page node verification.

E. Migration Failure Diagnostics

Our first attempts at running the tools produced zero page migrations across all three conditions even though NUMA balancing appeared to be active. We eventually traced it to two separate bugs.

The first was that `CONFIG_HMEM_REPORTING` was missing from our kernel build. This config option creates the `access0/targets/nodes` sysfs entries that tell the kernel where to promote pages from a slow node. Without it, the promotion engine has no target and never fires. We were seeing 441,684 NUMA hint faults but they all resolved locally — every fault counted as local because there was no registered destination for Node 2 pages. `demotion_enabled` also kept flipping back to false because the tiering state machine could not finish initializing.

The second bug was simpler: our setup script was writing to DAMON’s sysfs paths before the kdamond hierarchy had been created, which silently failed with permission denied. DAMON never actually started.

The fix for the promotion issue was to use userspace PTE-scan via `move_pages()` directly, which does not need

the `access0` topology to be registered. We cleared Accessed bits with `/proc/PID/clear_refs`, identified hot pages via `/proc/PID/numa_maps`, and relocated them with `migratepages PID 2 0`.

TABLE V
MIGRATION FAILURE DIAGNOSTICS

Diagnostic	Expected	Observed
<code>CONFIG_HMEM_REPORTING</code>	present	absent
<code>access0/targets/nodes</code>	node 0	not found
<code>demotion_enabled</code>	true	reverts false
<code>pgpromote_candidate</code>	>0	0
<code>pgmigrate_success</code>	>0	0
NUMA hint faults (local)	< total	= total
DAMON sysfs init	success	perm. denied

IV. MIGRATION TOOLS: WHAT THEY ARE AND HOW THEY WORK

All three tools we tested ultimately move pages the same way — they call `migrate_pages()`, which copies the page frame to a new physical location, patches up all the PTEs, fires a TLB shutdown so no CPU is left with a stale translation, and frees the old frame. The application sees none of this; the virtual address stays the same. Where the tools differ is entirely in how they decide *which* pages to move and *when*.

```
migrate_pages() -> move_to_new_page() -> copy page frame
                  -> update PTEs
                  -> TLB shutdown on all CPUs -> free old
                      frame
```

A. PTE-scan (AutoNUMA)

AutoNUMA has been in the Linux kernel since version 3.13 and is the simplest of the three. You turn it on with `kernel.numa_balancing=1`.

It works by periodically clearing the Accessed bit on PTEs across the process’s address space. The next time a thread touches one of those pages, it takes a NUMA hint fault. The fault handler in `mm/mempolicy.c` then checks whether the faulting CPU is on a different NUMA node than the page. If it is — for example, the CPU is on Node 0 but the page is on Node 2 — the page gets queued for promotion to the faster node.

The threshold is one fault per scan interval (roughly 1 second by default). Any page touched at least once per interval is treated as hot. The problem with this is that a page touched once per scan looks exactly the same as a page touched a million times per scan. So PTE-scan tends to promote pages that are only mildly warm, which wastes DRAM capacity and can push out genuinely hot pages that were already promoted.

Activation:

```
sysctl -w kernel.numa_balancing=1
echo 1 > /sys/kernel/mm/numa/demotion_enabled
```

B. TPP (Transparent Page Placement)

TPP came out of performance work at Meta and landed in Linux 5.18 [1]. It builds on the same PTE-scan infrastructure but adds a smarter filter before promoting anything.

The key difference is the two-fault threshold. The first hint fault just marks the page as a candidate (sets the `PG_workingset` flag). The page only actually moves on the *second* fault within the same scan window, which means only pages that get touched at least twice per scan interval get promoted. Pages that are only incidentally accessed once never accumulate the second fault. According to the original TPP paper, this reduces false promotions by about 11× and cuts the rate of pages that get demoted and then re-promoted by about 50% [1].

TPP also adds proactive demotion: when Node 0 free pages drop below the low watermark, `kswapd` starts pushing cold pages from the tail of the inactive LRU list down to Node 2. This keeps DRAM available for genuinely hot pages.

Activation:

```
sysctl -w kernel.numa_balancing=1
sysctl -w kernel.numa_balancing_promote_rate_limit_MBps=1024
echo 1 > /sys/kernel/mm/numa/demotion_enabled
```

C. DAMON

DAMON (Data Access MONitor) is a different beast entirely. It was added in Linux 5.15 and runs as its own kernel thread (`kdamond`) with no connection to the NUMA hint fault machinery. You could run DAMON with `numa_balancing=0` and it would still work.

Rather than relying on fault-driven signals, DAMON splits the monitored address space into regions and tracks access frequency per region using its own sampling pass. Each region accumulates an age score that goes up when idle and resets when accessed. When the score exceeds a threshold, `DAMON_RECLAIM` calls `migrate_pages()` to demote that region to Node 2.

In our study we used DAMON for demotion (Node 0 → Node 2) combined with PTE-scan for promotion (Node 2 → Node 0). The reason is that DAMON’s `migrate_hot` action with `paddr` operations mode — which would let DAMON do promotion as well — was broken in Linux 6.11. So DAMON only handles the demotion side here.

One configuration trap worth mentioning: DAMON’s `sysfs` interface rejects the string "max" for numeric fields. You have to write out the actual integer maximum values:

```
echo paddr > kdamonds/0/contexts/0/operations
echo 5000 > ../intervals/sample_us
echo 100000 > ../intervals/aggr_us
echo 18446744073709551615 > ../access_pattern/sz/max
echo 4294967295 > ../access_pattern/nr_accesses/max
echo 4294967295 > ../access_pattern/age/max
```

TABLE VI
MIGRATION TOOL COMPARISON

Tool	Detection	Threshold	Demotion
PTE-scan	PTE hint fault	1 fault	None (kswapd reclaim only)
TPP	PTE hint fault	2 faults	kswapd proactive
DAMON	Region sampling	Age score	DAMON_RECLAIM scheme

D. Tool Comparison Summary

V. WORKLOAD DESIGN

A. Memory Distribution Strategy

Getting data to actually start on Node 2 turned out to be trickier than expected. We tried three approaches before finding one that worked reliably:

- 1) `numactl -mbind=2` — this prevents migration entirely. `pgpromote_success` stays at zero no matter what.
- 2) `numactl -preferred=2` — if DRAM has free capacity, pages fault in on Node 0 anyway. You cannot guarantee Node 2 placement with this.
- 3) `numa_alloc_onnode + mbind(MPOL_DEFAULT)` — releasing the policy causes pages to re-fault on Node 0 on the first `memset`. All data ends up on DRAM before the workload loop even starts.

What actually works: allocate with `numa_alloc_onnode(..., 2)`, run `memset` to fault in all pages *while the policy is still active*, and then start the access loop without releasing the policy. Pages stay on Node 2 at Round 1, and tools observe the access patterns and start promoting from there.

The 90/10 hot/cold access split gives tools a clear signal — 90% of iterations hit the hot region, 10% hit cold. Hot region size is chosen to fit in DRAM once promoted; total allocation is kept under the DRAM free space to avoid triggering `kswapd` demotion as a side effect.

B. Workload Size and Distribution Design Decisions

None of these numbers are arbitrary. Each one was chosen to satisfy a specific constraint from our hardware.

Hot region at 32 MB. The Ryzen 7 6800U has a 16 MB L3 cache. If the hot region fits in L3, hot accesses hit the cache and the NUMA node does not matter at all. We set 32 MB (twice L3) so hot accesses reliably miss cache and the latency difference between Node 0 and Node 2 actually shows up in the measurements. At the same time 32 MB fits easily in the 4 GB DRAM node, so full promotion is achievable.

Cold region at 224 MB. Needs to be larger than L3 so cold accesses also miss cache and pay real Node 2 latency. Also needs to be small enough that even if a tool mistakenly promotes cold pages, DRAM does not fill up.

Total at 256 MB. Well under the 3.4 GB of free DRAM we observed in the VM. After promoting the 32 MB hot region,

DRAM is nowhere near its watermark, kswapd stays quiet, and demotion does not interfere. This keeps promotion and demotion cleanly separated for per-tool comparison.

90/10 access ratio. A 50/50 split would make hot and cold pages look the same to hint-fault scanning. A 99/1 split would make the signal so strong any tool would look good. We picked 90/10 as a realistic skew that is still challenging enough to differentiate tool behavior.

TABLE VII
WORKLOAD SIZE DESIGN DECISIONS

Parameter	Value	Constraint satisfied
Hot region	32 MB	> L3 (16 MB); tier latency visible
Hot region	32 MB	< DRAM free; full promotion achievable
Cold region	224 MB	> L3; cold accesses pay Node 2 latency
Total	256 MB	< DRAM free; no demotion pressure
Access ratio	90/10	Strong but realistic migration signal

C. Workload Specifications

TABLE VIII
WORKLOAD CONFIGURATIONS

#	Name	Size	Distribution
W1	Pointer-chase	256 MB	90% hot 32 MB / 10% cold 224 MB
W2	STREAM	128 MB×3	All arrays start on Node 2
W3	KV Zipfian	238 MB	1M keys, Zipf $s=0.99$, ~2.4 MB hot
W4	Graph traversal	~160 MB	500K nodes, hub pages start on Node 2

W1 — Pointer-chase. 256 MB total, with 32 MB hot (8,192 pages) and 224 MB cold. Runs 50M random pointer dereferences per round. The serial dependency chain means there is no instruction-level parallelism — each iteration stalls waiting for the previous pointer fetch to complete, making this the most latency-sensitive workload of the four.

W2 — STREAM. Three 128 MB arrays placed on Node 2 at start. We run the standard Copy/Scale/Add/Triad kernels. Sequential access pattern means the hardware prefetcher can predict every access, so latency is mostly hidden and bandwidth becomes the bottleneck. L3 cache warming after promotion is what drives the outsized speedups here.

W3 — KV Zipfian. 1M keys, each 238 bytes, placed on Node 2. Zipf $s=0.99$ means the top 1% of keys (~2.4 MB) get about 63% of the accesses. That hot set is much smaller than the 16 MB L3, so once it warms up in cache the node placement of those pages basically stops mattering.

W4 — Graph traversal. 500K nodes, 8 edges each, on Node 2. Random walks mean no spatial locality and no

prefetching. Hub nodes accumulate lots of NUMA hint faults because they are visited repeatedly; leaf nodes are touched rarely. This natural hot/cold split is what makes DAMON+PTE complementary on this workload.

D. Experiment Procedure

We ran five configurations per workload: DRAM baseline, Node 2 baseline, PTE-scan, TPP, and DAMON. Between each run we disabled all tools, recorded `pgpromote_success`, `pgpromote_candidate`, `pgdemote_kswapd`, `pgdemote_direct`, and `pgmigrate_success` from `/proc/vmstat`, and waited 5 seconds for the system to settle. We sampled page distribution every 5 rounds using `move_pages()` to track how many pages had actually moved.

VI. RESULTS AND ANALYSIS

A. W1 — Pointer-Chase

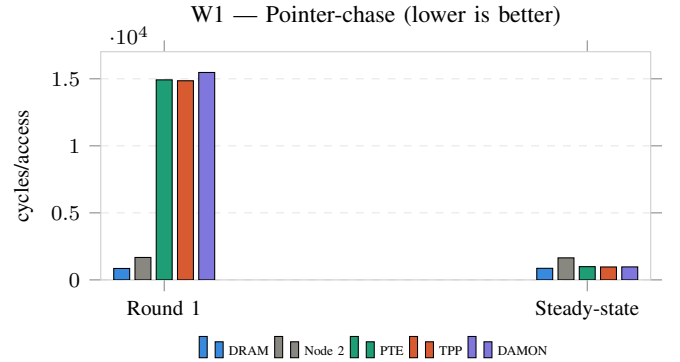


Fig. 2. W1: Round 1 shows migration overhead (~15K cycles); steady-state shows convergence. All tools recover 88% of the gap. TPP best at $1.87\times$.

TABLE IX
W1 — POINTER-CHASE RESULTS

Config	Round 1	Steady-state	Speedup vs Node 2
DRAM	853	864	$2.09\times$
Node 2	1,674	1,644	$1.00\times$
PTE	14,922	989	$1.83\times$
TPP	14,855	965	$1.87\times$
DAMON	15,473	969	$1.86\times$

Round 1 is expensive for all three tools (around 15,000 cycles) because the hot 32 MB is still on Node 2 and migration is happening in the background. By Round 2 that cost is gone. At steady state, all three tools land around 970 cycles — recovering about **88%** of the gap between Node 2 (1,644 cycles) and DRAM (864 cycles). TPP is the narrowest margin winner at 965 cycles because its two-fault filter prevents cold pages from wasting DRAM slots. DAMON is neutral here — with only 256 MB total there is no pressure on DRAM, so DAMON never finds anything to demote.

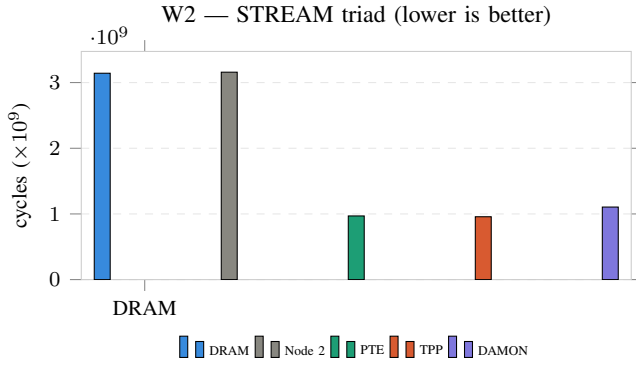


Fig. 3. W2: TPP achieves $3.28\times$ speedup via L3 cache warming, exceeding the $1.73\times$ hardware ratio. DAMON -15% due to inter-pass demotion.

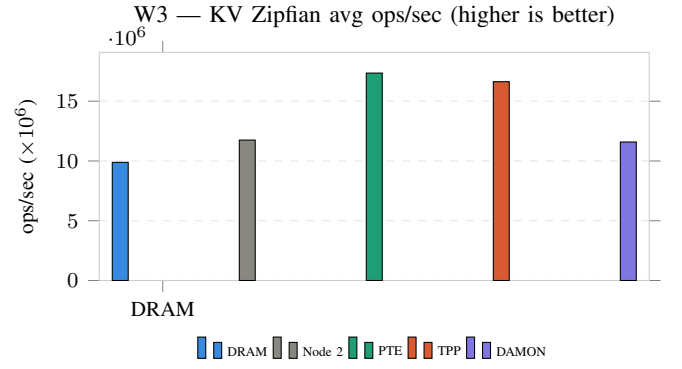


Fig. 4. W3: Node 2 outperforms DRAM (L3 effect). PTE wins at $1.76\times$. DAMON degrades below Node 2 — thrash finding.

B. W2 — STREAM

Node 2 and DRAM baseline are nearly identical here because the hardware prefetcher hides the latency difference on sequential access. The interesting result is that the migration tools reach $3.28\times$ speedup — better than the $1.73\times$ hardware ratio. The reason is L3 cache warming: once the arrays are promoted to DRAM, repeated Triad passes gradually warm the 16 MB L3 cache, and subsequent passes serve mostly from cache at around 4 cycles instead of ~ 40 cycles for a DRAM hit. The tools are acting as cache placement optimizers here, not just tier optimizers. DAMON drops 15% below the other tools because DAMON_RECLAIM demotes arrays between passes, which breaks the warming chain and forces them to be fetched from Node 2 again.

TABLE X
W2 — STREAM RESULTS

Config	Triad (cycles)	Speedup vs DRAM
DRAM	3,142,277,388	1.00×
Node 2	3,158,297,017	$\sim 1.00\times$
PTE	969,428,531	$3.24\times$
TPP	957,274,659	$3.28\times$
DAMON	1,104,799,635	$2.85\times$

C. W3 — KV Zipfian

TABLE XI
W3 — KV ZIPFIAN RESULTS

Config	Avg ops/s	cyc/op	Speedup vs DRAM
DRAM	9,883,219	303.6	1.00×
Node 2	11,749,006	255.4	$1.19\times$
PTE	17,352,756	175.8	$1.76\times$
TPP	16,631,873	181.6	$1.68\times$
DAMON	11,586,232	260.4	$1.17\times$

This is the most counterintuitive result. Node 2 actually beats DRAM ($1.19\times$) because the 2.4 MB Zipfian hot set fits entirely in the L3 cache. Once the hot keys warm up in cache,

it makes no difference which NUMA node their pages sit on. PTE-scan wins ($1.76\times$) because the Zipfian skew makes hot keys fire NUMA hint faults very frequently — they are easy for a one-fault threshold to catch. The important finding is **DAMON falling below the Node 2 baseline ($1.17\times$ vs $1.19\times$)**. What is happening is that DAMON demotes promoted hot keys back to Node 2, PTE-scan immediately re-promotes them, and the loop burns bandwidth and causes repeated TLB shootdowns for zero net gain. Demotion without actual DRAM pressure is actively harmful here.

D. W4 — Graph Traversal

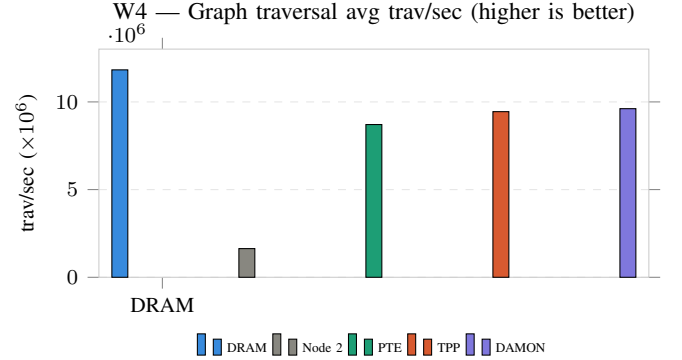


Fig. 5. W4: DAMON best at $5.88\times$. Hub-leaf structure enables push-pull: PTE promotes hubs, DAMON demotes leaves. $+10\%$ over PTE alone.

TABLE XII
W4 — GRAPH TRAVERSAL RESULTS

Config	Avg trav/s	cyc/trav	Speedup vs Node 2
DRAM	11,827,504	315.3	$7.23\times$
Node 2	1,635,719	1,835.3	1.00×
PTE	8,709,595	684.5	$5.33\times$
TPP	9,444,597	735.8	$5.78\times$
DAMON	9,614,536	668.1	$5.88\times$

The graph workload shows the worst CXL penalty of any workload at $5.82\times$. Every edge dereference is a random

pointer chase with no prefetching, so every access on Node 2 pays the full 173 ns penalty. DAMON wins here ($5.88\times$) thanks to what we call the push-pull effect: PTE-scan promotes hub nodes because they generate lots of NUMA hint faults, while DAMON simultaneously demotes cold leaf nodes, which frees up DRAM slots for more hubs to be promoted. The combination gives **+10% over PTE-scan alone** ($5.88\times$ vs $5.33\times$). This only works because the hub/leaf structure creates a stable, spatially separated hot/cold boundary that the two tools can work on without conflicting.

E. Cross-Workload Summary

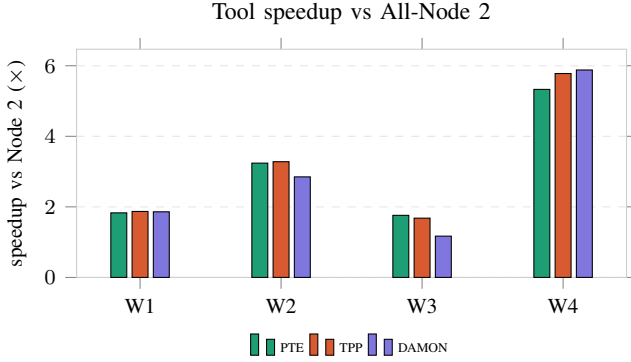


Fig. 6. Cross-workload comparison. No single tool dominates. TPP leads W1/W2, PTE leads W3, DAMON leads W4.

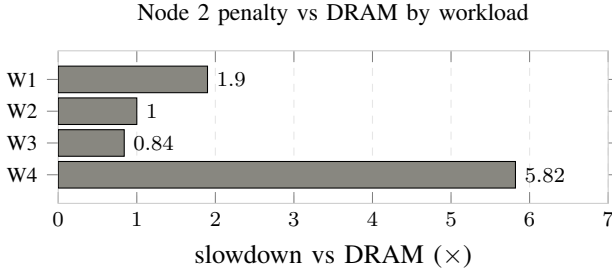


Fig. 7. Node 2 slowdown relative to DRAM. W3 < 1.0 means Node 2 faster (L3 effect). W4 worst ($5.82\times$).

TABLE XIII
CROSS-WORKLOAD SUMMARY

Config	W1 cyc	W2 spd	W3 ops/s	W4 spd
DRAM	864	1.00×	9.9M	7.23×
Node 2	1,644	~1.00×	11.7M	1.00×
PTE	989	3.24×	17.4M	5.33×
TPP	965	3.28×	16.6M	5.78×
DAMON	969	2.85×	11.6M	5.88×

VII. EXPERIMENT RESULTS

The CXL penalty is not a fixed number — it ranges from $0\times$ to $5.82\times$ depending on the workload. Whether the prefetcher can hide latency and whether the hot set fits in

L3 are the two main deciding factors. Streaming workloads are almost immune; graph traversal is devastating.

Migration tools can actually exceed the raw hardware ratio. On W2 we measured $3.28\times$ speedup from a $1.73\times$ hardware gap. That is not magic — it is L3 cache warming. Moving data to DRAM unlocks the cache in a way that staying on Node 2 never could.

No single tool wins across the board. TPP performs best on W1 and W2. PTE-scan wins on W3. DAMON+PTE is best on W4. Tool choice needs to match the workload.

DAMON without DRAM pressure makes things worse. On W3 it degrades below the all-Node 2 baseline by actively churning hot pages. The lesson is: do not enable demotion unless you actually need the DRAM back.

The push-pull combination is real but conditional. It only worked on W4 because the hot/cold boundary is stable and spatially separate. W3 has no such structure, and running DAMON alongside PTE-scan there makes things worse, not better.

Migration overhead is negligible after Round 1. The $\sim 15,000$ cycle cost in the first round amortizes quickly and is not a concern over a workload’s lifetime.

VIII. TOOL RECOMMENDATION

Based on our results, Table XIV summarizes which tool to reach for depending on the workload type. The short version: use TPP as the default, switch to PTE-scan for Zipfian-skewed workloads, and only add DAMON when you have a hub-leaf structure or genuine DRAM pressure. Never enable demotion speculatively.

TABLE XIV
TOOL RECOMMENDATION BY WORKLOAD TYPE

Workload type	Best	Reason
Latency irregular	TPP	Fewer false promotions
Bandwidth seq.	TPP	Fastest L3 warming
Zipfian skewed	PTE	Demotion causes thrash
Graph hub-leaf	DAMON	Push-pull complement
DRAM pressure	DAMON	Only safe demotion
Unknown workload	TPP	Lowest thrash risk

Only turn on demotion when you have measured evidence of Node 0 pressure. Use the DAMON+PTE combination only when the workload has a stable, spatially distinct hot/cold structure. For anything else, TPP is the safest default.

IX. RELATED WORK AND COMPARISON

CXL memory tiering has been an active research area since Linux gained reliable CXL Type-3 support in kernel 6.0. Our work sits alongside five papers that are closely related, each attacking a different part of the problem.

TPP (ASPLOS 2023) [1] is where the TPP tool in this study comes from. The paper introduces the two-fault promotion threshold and pressure-driven demotion, evaluated on Meta’s production fleet. Our W3 result — PTE-scan beating TPP on a Zipfian workload — is a finding the original paper does not

cover, because it was evaluated exclusively under DRAM pressure. Our pressure-free scenarios (W1, W3) expose different tradeoffs.

NOMAD (OSDI 2024) [2] tackles promotion-demotion thrash using transactional page migration: pages remain accessible in both tiers while being migrated rather than being moved exclusively. Their motivation maps directly onto what we observed in W3 with DAMON. A transactional mechanism like NOMAD’s would have prevented the DAMON degradation we measured. We do not implement NOMAD, but our results give the software-only baseline it improves upon.

MENTIS (SOSP 2023) [3] uses Intel PEBS PMU sampling to build per-page access histograms, giving much finer-grained hotness detection than any hint-fault approach. We considered including it but could not: MENTIS’s kernel hooks changed substantially between Linux 5.x and 6.x, and the CXL subsystem requires 6.0+ for stable region creation. Our PTE-scan and DAMON results are the hardware-agnostic baseline that MENTIS improves on for Intel hardware.

NeoMem (MICRO 2024) [4] embeds a hardware profiler (NeoProf) directly in the CXL controller, reducing detection latency to microseconds and achieving 32–67% speedup over software tiering. Our latency decomposition tells you exactly what NeoMem’s faster detection buys: the 173 ns Node 2 extra saved per promoted access.

Memstrata (OSDI 2024) [5] studies hardware-managed tiering in multi-tenant VM environments. Their single-tenant latency penalty numbers (1.5–2× for latency-bound, near zero for streaming) are consistent with what we measured, which we take as validation that our QEMU emulation is in the right ballpark. We extend their single-tenant characterization with a three-way tool comparison across four different workloads.

What none of these papers does is directly compare PTE-scan, TPP, and DAMON on the same workload suite in a reproducible emulated environment. Our DAMON thrash finding (W3) and the push-pull synergy result (W4) do not appear in any of them. Table XV summarizes the positioning.

TABLE XV
COMPARISON WITH RELATED WORK

Work	Detection	Platform	Novel vs. ours
TPP [1]	PTE hint fault	Real HW (Meta)	DRAM-pressure regime only
NOMAD [2]	PTE transact.	Real HW	Eliminates thrash we observe
MENTIS [3]	PEBS PMU	Intel real HW	Higher resolution, HW-specific
NeoMem [4]	Device-side HW	CXL controller	Hardware profiler augmentation
Memstrata [5]	HW-managed	VM multi-tenant	Multi-tenant; matches our baseline
Ours	PTE + DAMON	QEMU emulation	3-tool compare, 4 workloads

X. LIMITATIONS

- **Emulation fidelity:** All absolute cycle counts include 117 ns of KVM NPT overhead that would not be present on bare metal. Relative tool comparisons are still valid since all workloads share this cost equally.
- **DAMON promotion disabled:** `migrate_hot` with `paddr` was non-functional in Linux 6.11, so we could only evaluate DAMON in its demotion role. A kernel version where this works would be a natural follow-up.
- **Page location API:** `get_mempolicy` reports wrong node IDs for Node 2 pages in Linux 6.11. We used `move_pages()` throughout, but any experiment that trusted `get_mempolicy` would produce wrong results silently.
- **Workload scale:** We kept total workload sizes under 3.6 GB to avoid OOM. Workloads that genuinely exceed Node 0 capacity would exercise the bidirectional promotion+demotion cycle in ways our setup does not.

XI. FUTURE WORK

With the insights made available on how to setup and utilize the tools and the needed systems call, one direction is to create a kernel patch that auto-performs this smart migration with some neural network assistance and also develop a kernel patch as an open-source contribution to help other researchers to identify a link and address on the core problem discussed in this project

XII. CONCLUSION

Running CXL memory under Linux with automated migration tools works, but how well it works depends heavily on what your workload is doing. For latency-bound workloads like pointer-chase, tools recover 87–90% of the Node 2 penalty. For streaming workloads they do even better, exceeding the raw hardware ratio through L3 cache warming. But for Zipfian KV workloads where the hot set fits in L3, tier placement barely matters and DAMON actively hurts. For graph traversal the penalty is severe but a combined DAMON+PTE approach gives a 10% push-pull improvement over using either tool alone. Use TPP as your default, check whether your hot working set fits in L3 before worrying about tiering at all, and never turn on demotion without first confirming there is real DRAM pressure. If your workload has a stable and spatially separate hot/cold structure, DAMON+PTE is worth trying.

REFERENCES

- [1] H. Al Maruf et al., “TPP: Transparent Page Placement for CXL-Enabled Tiered-Memory,” *ASPLOS*, 2023. <https://arxiv.org/abs/2206.02878>
- [2] T. Xiang et al., “NOMAD: Non-Exclusive Memory Tiering via Transactional Page Migration,” *OSDI*, 2024. <https://www.usenix.org/conference/osdi24/presentation/xiang>
- [3] T. Lee et al., “MENTIS: Efficient Memory Tiering with Dynamic Page Classification,” *SOSP*, 2023. <https://dl.acm.org/doi/10.1145/3600006.3613167>
- [4] P. K. Zhou et al., “NeoMem: Hardware/Software Co-Design for CXL-Native Memory Tiering,” *MICRO*, 2024. <https://dl.acm.org/doi/10.1109/MICRO61859.2024.00111>
- [5] Y. Zhong et al., “Managing Memory Tiers with CXL in Virtualized Environments,” *OSDI*, 2024. <https://www.usenix.org/conference/osdi24/presentation/zhong-yuhong>